UDP Sockets Overview

Certainly. This excerpt introduces the basic concept of **UDP sockets** and how they are used in client-server applications. Let me explain it step by step, covering both the conceptual differences from TCP and the practical implementation via code.

8.1 Introduction to UDP Sockets

UDP (User Datagram Protocol) is **connectionless** and **unreliable**, in contrast to TCP, which is **connection-oriented** and **reliable**. These characteristics have major implications on how network programs are written:

- **TCP** requires a handshake before communication begins and ensures that packets arrive in order and without errors.
- **UDP** sends datagrams without setting up a connection and does **not** guarantee delivery, order, or error-checking at the transport level.

Despite this, **UDP is useful** in cases where speed is more critical than reliability, or where the application itself implements its own form of reliability. Common applications using UDP include:

- **DNS** (Domain Name System)
- **NFS** (Network File System)
- **SNMP** (Simple Network Management Protocol)

Typical UDP Client/Server Interaction (Figure 8.1)

- A **UDP client** uses the `sendto()` function to send a message directly to the server.
- A **UDP server** uses the `recvfrom()` function to wait for incoming datagrams.

There is **no connection establishment** as in TCP. Therefore:

- The server does not use `listen()` or `accept()`.
- The client does not use `connect()` to establish a session.

Instead:

- The **client must supply the server's address** with every message using `sendto()`.
- The **server learns the client's address** from the return value of `recvfrom()`, which allows it to send a reply.

8.2 `recvfrom()` and `sendto()` Functions

These are the core I/O functions for datagram sockets (used in UDP communication):

Function Signatures

```c
#include <sys/socket.h>

ssize_t recvfrom(int sockfd, void *buff, size_t nbytes, int flags,
                 struct sockaddr *from, socklen_t *addrlen);

ssize_t sendto(int sockfd, const void *buff, size_t nbytes, int flags,
               const struct sockaddr *to, socklen_t addrlen);
```

Parameters Breakdown

- `sockfd`: File descriptor returned by `socket()`.
- `buff`: Pointer to the buffer where data will be read into (`recvfrom`) or written from (`sendto`).
- `nbytes`: Number of bytes to read/write.
- `flags`: Usually set to `0` for simple applications.
- `from`/`to`: Pointers to a `sockaddr` structure (such as `sockaddr_in`) containing the sender's or recipient's address.
- `addrlen`: Size of the address structure.

Key Differences:

- `recvfrom` provides the **sender's address** through `from` and `addrlen`.
- `sendto` requires the **receiver's address** through `to` and `addrlen`.

If you're not interested in knowing who sent the data in `recvfrom()`, you can pass `NULL` for `from` and `addrlen`.

Return Values

- Both functions return the **number of bytes sent/received**.
- A return value of `0` from `recvfrom()` is **valid** in UDP (it just means an empty datagram).
- This is unlike TCP, where `read()` returning 0 indicates **connection closure**.

8.3 UDP Echo Server – Main Function Explained

Now, let's look at how a **simple UDP echo server** is implemented.

```c
#include "unp.h"

int main(int argc, char **argv) {
    int sockfd;
    struct sockaddr_in servaddr, cliaddr;

    sockfd = Socket(AF_INET, SOCK_DGRAM, 0);
    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
```

```
    servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
    servaddr.sin_port = htons(SERV_PORT);

    Bind(sockfd, (SA *) &servaddr, sizeof(servaddr));
    dg_echo(sockfd, (SA *) &cliaddr, sizeof(cliaddr));
}
```

Explanation Line by Line

Line 7:

```c
sockfd = Socket(AF_INET, SOCK_DGRAM, 0);
```

- A socket is created with:
  - `**AF_INET**`: IPv4 address family.
  - `**SOCK_DGRAM**`: Indicates UDP (datagram) socket.
  - `**0**`: Default protocol (UDP in this case).

Line 8–11:

```c
bzero(&servaddr, sizeof(servaddr));
servaddr.sin_family = AF_INET;
servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
servaddr.sin_port = htons(SERV_PORT);
```

- The server's socket address (`**servaddr**`) is being filled:
  - Set all fields to zero with `**bzero**`.
  - Set address family to IPv4.
  - Set IP to `**INADDR_ANY**`, meaning the server listens on all interfaces.
  - Set the port to a predefined constant `**SERV_PORT**`.

Line 12:

```c
Bind(sockfd, (SA *) &servaddr, sizeof(servaddr));
```

- Binds the socket to the local IP address and port.
- This is necessary for the server to receive incoming datagrams.

Line 13:

```c
dg_echo(sockfd, (SA *) &cliaddr, sizeof(cliaddr));
```

- This is the core function where the server enters a loop:
  - Waits for data using `**recvfrom**`.
  - Sends back the received data using `**sendto**`.
  - Uses `**cliaddr**` to store the address of the client for response.

Summary of the Server Flow

1. **Create a UDP socket.**
2. **Bind it to a specific port.**
3. **Enter a loop to:**
   - Wait for incoming datagrams from clients.
   - Read the message using `**recvfrom**`.
   - Send the same message back to the client using `**sendto**`.

There is **no connection setup or teardown** with UDP. It's all stateless and message-based.

Let me know if you'd like to explore the `**dg_echo()**` function or the UDP client code next.

This explanation covers a **UDP echo client-server model** using the `**dg_echo**` (server-side) and `**dg_cli**` (client-side) functions from the book *UNIX Network Programming* by W. Richard Stevens.

Section 8.4: UDP Echo Server – `dg_echo` Function

Code Breakdown

```c
void dg_echo(int sockfd, SA *pcliaddr, socklen_t clilen)
{
    int n;
    socklen_t len;
    char mesg[MAXLINE];
    for ( ; ; ) {
        len = clilen;
        n = Recvfrom(sockfd, mesg, MAXLINE, 0, pcliaddr, &len);
        Sendto(sockfd, mesg, n, 0, pcliaddr, len);
    }
}
```

Detailed Explanation

- **Line 3:** `**dg_echo**` is a server-side function that handles datagrams.
  - It takes:

    - `sockfd`: the socket descriptor of the UDP socket,
    - `pcliaddr`: pointer to a socket address structure to hold the client's address,
    - `clilen`: the length of that address structure.
- **Line 7:** `mesg[MAXLINE]` defines the buffer to receive and send messages.
- **Infinite Loop (Lines 8–12):**
  - **Line 9:** Set `len` to `clilen`, which prepares to receive the client's address.
  - **Line 10:** `Recvfrom` is used to read a datagram from the UDP socket.
    - It fills `mesg` with the incoming data,
    - `pcliaddr` is updated with the sender's address.
  - **Line 11:** `Sendto` sends the received message back to the same address stored in `pcliaddr`.

Key Concepts

- **Iterative Server:** The function handles one client at a time. There is **no** `fork`, meaning the server doesn't create new processes or threads for each client.
- **Connectionless Protocol:** UDP does **not** establish or maintain a connection. It just sends/receives packets. Hence, the server runs in an infinite loop waiting for incoming packets.
- **Socket Buffering and Queuing:**
  - UDP sockets use an internal **receive buffer**.
  - Incoming datagrams are stored in this buffer (FIFO order) until `recvfrom` is called.
  - If datagrams arrive faster than they are processed, they can be **dropped** once the buffer is full.
  - You can configure the buffer size using `SO_RCVBUF`.
- **Protocol-Independent Design:**
  - `dg_echo` does **not access the contents** of the `sockaddr` structure directly.
  - It relies on the caller (usually `main`) to provide the correct socket address structure.
  - `recvfrom` and `sendto` are responsible for interpreting that address information.

Section 8.5: UDP Echo Client – `main` Function

Code Breakdown

```c
int main(int argc, char **argv)
{
    int sockfd;
    struct sockaddr_in servaddr;

    if (argc != 2)
        err_quit("usage: udpcli <IPaddress>");

    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
```

```
    servaddr.sin_port = htons(SERV_PORT);
    Inet_pton(AF_INET, argv[1], &servaddr.sin_addr);

    sockfd = Socket(AF_INET, SOCK_DGRAM, 0);
    dg_cli(stdin, sockfd, (SA *) &servaddr, sizeof(servaddr));

    exit(0);
}
```

Detailed Explanation

- **Lines 7–12:**
  - Check if the user provided the server IP address.
  - Initialize the `**servaddr**` structure:
    - Set address family to IPv4,
    - Set the port to the predefined `**SERV_PORT**`,
    - Convert the string IP address to binary form using `**inet_pton**`.
- **Line 13:**
  - Create a **UDP socket** with `**SOCK_DGRAM**`.
- **Line 14:**
  - Call `**dg_cli**`, passing standard input, socket descriptor, and server address info.

Section 8.6: UDP Echo Client – `dg_cli` Function

Code Breakdown

```c
void dg_cli(FILE *fp, int sockfd, const SA *pservaddr, socklen_t servlen)
{
    int n;
    char sendline[MAXLINE], recvline[MAXLINE + 1];

    while (Fgets(sendline, MAXLINE, fp) != NULL) {
        Sendto(sockfd, sendline, strlen(sendline), 0, pservaddr, servlen);
        n = Recvfrom(sockfd, recvline, MAXLINE, 0, NULL, NULL);
        recvline[n] = 0;  // Null-terminate the received string
        Fputs(recvline, stdout);
    }
}
```

Detailed Explanation

- **Client I/O Loop (Lines 7–12):**
  - Read a line from standard input.
  - Send the line to the server using `**sendto**`.
  - Receive a response using `**recvfrom**`.
  - Output the echoed message to standard output.

- **recvfrom and Socket Binding:**
  - When a UDP socket is created, it is **not** bound to a local port.
  - The **first call to** `sendto` causes the kernel to bind an ephemeral (temporary) port.
  - `recvfrom` uses `NULL` for address fields since the client does not need to know the sender.
- **Security Note:**
  - Since there is **no verification of the sender**, any host can send a response.
  - The client will assume it came from the correct server.
  - This opens up **spoofing risks**, which will be discussed further in later sections (e.g., 8.8).
- **Protocol Independence:**
  - Like `dg_echo`, `dg_cli` is protocol-independent.
  - It accepts a pointer to a socket address structure but does not look inside it.

Section 8.7: Lost Datagrams

Key Issues in UDP Reliability

- **No Guarantee of Delivery:**
  - If a client's datagram is lost (e.g., dropped by a router), the server will never receive it.
  - The client, waiting on `recvfrom`, will block **forever**.
- **Reply Loss:**
  - If the server replies but that response is lost, the client again blocks forever.

Workarounds

- **Timeout Mechanism:**
  - Set a **timeout on** `recvfrom` so that the client doesn't block indefinitely.
  - This gives a chance to **retransmit** the request.
- **Ambiguity of Failure:**
  - A timeout alone doesn't reveal **where** the packet was lost:
    - Did the request fail to reach the server?
    - Or did the server's response not make it back?
- **Critical Scenarios:**
  - In applications involving transactions (e.g., money transfer), knowing which packet was lost is crucial.
  - Blindly retrying can lead to **duplicate actions** if the server received and processed the request, but the client didn't get the reply.

Conclusion

This UDP echo server-client design demonstrates:

- The **stateless and connectionless nature of UDP**,
- The **simplicity** and **efficiency** of using a single-threaded iterative server,
- The **importance of socket buffering and receive buffers**,
- The **differences between TCP and UDP** in terms of connections, reliability, and data delivery.

While this example works for basic echoing, it highlights the challenges with **building reliable communication** over UDP. These must be addressed through application-layer mechanisms like timeouts, sequence numbers, and acknowledgments.

Certainly. Let's go through the detailed explanation of the Domain Name System (DNS) as outlined in the provided text.

11.2 Domain Name System (DNS) – Detailed Explanation

What is DNS?

The **Domain Name System (DNS)** is a fundamental part of the internet infrastructure. Its primary job is to **map hostnames to IP addresses** and vice versa. This allows users and applications to refer to servers using readable names (like `www.google.com`) instead of numerical IP addresses (like `142.250.64.132`).

- A **hostname** can be:
  - A simple name like `solaris` or `freebsd`
  - A **Fully Qualified Domain Name (FQDN)** like `solaris.unpbook.com`

Fully Qualified Domain Name (FQDN)

An **FQDN** is:

- Also called an **absolute name**
- Technically must end with a **trailing period (.)**, for example: `solaris.unpbook.com.`
- However, users often omit this final period
- The trailing period signals to the DNS resolver that this name is already fully qualified and doesn't require domain suffixes to be appended during a search

---

Resource Records (RRs)

The DNS system stores data as **Resource Records (RRs)**. Each RR maps some kind of information to a domain name.

Types of DNS Resource Records Relevant to Network Programming:

1. A Record

- Maps a hostname to a **32-bit IPv4 address**
- Example:

```nginx
freebsd IN A 12.106.32.254
```

- Here, the hostname `freebsd` in the domain `unpbook.com` is assigned the IPv4 address `12.106.32.254`.

2. AAAA Record (Quad A)

- Maps a hostname to a **128-bit IPv6 address**
- Example:

```ruby
IN AAAA 3ffe:b80:1f8d:1:a00:20ff:fea7:686b
```

- This is four times the size of an A record and used for IPv6.

3. PTR Record (Pointer Record)

- **Reverses the mapping**: IP address to hostname
- For **IPv4**:
  - The four bytes are reversed and represented as ASCII decimal values, followed by `.in-addr.arpa`
  - Example: `12.106.32.254` becomes `254.32.106.12.in-addr.arpa`
- For **IPv6**:
  - The 128-bit address is split into 32 nibbles (4-bit segments)
  - These are reversed, converted to hex, and appended to `.ip6.arpa`
  - Example:

```css
b.6.8.6.7.a.e.f.f.f.0.2.0.0.a.0.1.0.0.0.d.8.f.1.0.8.b.0.e.f.f.3.ip6.arpa
```

Older standards used `ip6.int`, but this has been replaced with `ip6.arpa` for consistency.

4. MX Record (Mail Exchanger)

- Specifies a **mail server** for a domain
- Includes a **preference value** to determine order of usage
- Example:

```
nginx


IN MX 5 freebsd.unpbook.com.
IN MX 10 mailhost.unpbook.com.
```

- The mail system will try the lowest-numbered preference first

5. CNAME Record (Canonical Name)

- Used to define **aliases** for domain names
- Commonly used for services like `**ftp**`, `**www**`, etc.
- Example:

```
objectivec


ftp IN CNAME linux.unpbook.com.
www IN CNAME linux.unpbook.com.
```

- Allows flexibility in server migration or maintenance

Multiple Record Variants for IPv4 and IPv6

In some deployments, administrators use naming conventions to differentiate address types:

- `**aix**` is a hostname
- `**aix-4**` might refer to the IPv4 address (A record)
- `**aix-6**` to the IPv6 global address (AAAA record)
- `**aix-611**` to the link-local IPv6 address (also an AAAA record)

Example:

```
ruby


aix      IN A     192.168.42.2
         IN AAAA  3ffe:b80:1f8d:2:204:acff:fe17:bf38
         IN MX    5 aix.unpbook.com.
         IN MX    10 mailhost.unpbook.com.
aix-4    IN A     192.168.42.2
aix-6    IN AAAA  3ffe:b80:1f8d:2:204:acff:fe17:bf38
aix-611 IN AAAA  fe80::204:acff:fe17:bf38
```

Resolvers and Name Servers

Name Server

- A **server that answers DNS queries** either from its local database or by querying other DNS servers
- Common implementation: **BIND (Berkeley Internet Name Domain)**

Resolver

- The **client-side library or daemon** that an application uses to query DNS
- Examples of resolver functions:
  - `gethostbyname` – maps hostname to IPv4 address
  - `gethostbyaddr` – maps IP address to hostname

Working Mechanism:

1. **Application** calls a resolver function
2. **Resolver** checks system config (usually `/etc/resolv.conf`) for the IP addresses of DNS servers
3. **Query is sent** to the local DNS server (typically via **UDP**)
4. If local server cannot answer:
   - It **forwards** the query across the Internet
5. If the **response is too large** for UDP:
   - The resolver **switches to TCP**

DNS Configuration Files

Common Files for DNS Configuration on Unix-like Systems:

System	File Used for Name Service Configuration
Solaris 2.x, HP-UX 10+, FreeBSD 5.x+	`/etc/nsswitch.conf`
AIX	`/etc/netsvc.conf`
BIND 9.2.2 (IRS System)	`/etc/irs.conf`

- **All** use `/etc/resolv.conf` to specify the DNS server IPs

  Note: Using hostnames in `/etc/resolv.conf` would create a paradox (chicken-and-egg problem) because DNS is needed to resolve the names of DNS servers themselves.

DNS Alternatives

DNS is not the only mechanism for name resolution. Alternatives include:

1. **Static Host Files**
   - Usually located at `/etc/hosts`
   - Manually maintained, suitable for small setups
2. **Network Information System (NIS)**
   - Centralized name service system often used in Unix environments
3. **LDAP (Lightweight Directory Access Protocol)**

- Directory-based protocol that can also provide naming and addressing info

4. **IRS (Information Retrieval Service)**
   - Provided with some BIND distributions

Summary

The DNS system:

- Translates human-readable hostnames into IP addresses and vice versa
- Uses different types of Resource Records for IPv4, IPv6, reverse mapping, mail routing, and aliases
- Is accessed by applications via resolver functions like `gethostbyname`
- Can fall back to TCP when necessary due to size limits in UDP
- Has flexible configuration mechanisms depending on the operating system
- Can be supplemented or replaced by static files or directory services in some environments

Let me know if you want a diagrammatic summary or practical examples of using DNS in C or Python programs.

Certainly. Below is a **detailed explanation of both the theory and the C code** presented in the `hostent.c` example that uses the `gethostbyname()` function to resolve hostnames into IP addresses.

---

1. THEORETICAL OVERVIEW

Purpose of `gethostbyname()`

The function `gethostbyname()` is used in network programming to **resolve a human-readable hostname (like `www.google.com`) to an IPv4 address**. This is useful because users and applications typically reference servers by their domain names, not raw IP addresses.

- It performs a **DNS A record lookup** (IPv4 only).
- It returns a pointer to a `struct hostent`, which contains:
  - The **canonical name** of the host
  - A list of **aliases**
  - The **address type** (typically `AF_INET`)
  - The **length** of the address in bytes
  - A list of **IP addresses** associated with the host

Why it is outdated

- `gethostbyname()` only supports **IPv4**.
- It is not **thread-safe**.
- It is marked as **obsolete** in modern POSIX systems.

- `getaddrinfo()` is preferred for modern code because:
  - It supports both **IPv4 and IPv6**
  - It is **reentrant and thread-safe**

---

2. STRUCTURE DEFINITION

Here's the definition of `struct hostent`:

```c
struct hostent {
    char    *h_name;        // Canonical (official) hostname
    char    **h_aliases;    // Null-terminated array of alias names
    int     h_addrtype;     // Address type (e.g., AF_INET)
    int     h_length;       // Length of each address (4 bytes for IPv4)
    char    **h_addr_list;  // Null-terminated array of IPv4 addresses
};
```

- `h_name`: Canonical name, possibly resolved via CNAME record.
- `h_aliases`: Other names associated with the host.
- `h_addrtype`: Typically `AF_INET`.
- `h_length`: Always 4 for IPv4 (since an IPv4 address is 4 bytes).
- `h_addr_list`: A null-terminated array of IP addresses. Each address is a `struct in_addr *` (pointer to a 4-byte binary IP address).

---

3. ERROR HANDLING WITH `h_errno`

Unlike most standard library functions that use `errno`, `gethostbyname()` sets a separate variable `h_errno` on error. Common values include:

- `HOST_NOT_FOUND`: Host not found
- `TRY_AGAIN`: Temporary failure, retry later
- `NO_RECOVERY`: A non-recoverable error occurred
- `NO_DATA`: Valid hostname but no A record (e.g., only MX record exists)

You can get human-readable strings for these using `hstrerror(h_errno)`.

---

4. CODE WALKTHROUGH

```c
#include "unp.h"                    // Custom header, likely includes standard headers and wrappe
int main(int argc, char **argv) {
    char *ptr, **pptr;
    char str[INET_ADDRSTRLEN];    // To hold string version of IPv4 address
    struct hostent *hptr;

    while (--argc > 0) {          // Loop over all command-line arguments
```

```
        ptr = *++argv;            // Get next hostname

        if ((hptr = gethostbyname(ptr)) == NULL) {
            err_msg("gethostbyname error for host: %s: %s",
                    ptr, hstrerror(h_errno)); // Error reporting
            continue;
        }

        printf("official hostname: %s\n", hptr->h_name);  // Print canonical name

        for (pptr = hptr->h_aliases; *pptr != NULL; pptr++)
            printf("\talias: %s\n", *pptr);              // Print each alias

        switch (hptr->h_addrtype) {
        case AF_INET:
            pptr = hptr->h_addr_list;                  // Point to list of IP addresses
            for (; *pptr != NULL; pptr++) {
                printf("\taddress: %s\n",
                       inet_ntop(hptr->h_addrtype, *pptr, str, sizeof(str)));
                // Convert binary IP to string format and print
            }
            break;
        default:
            err_ret("unknown address type");
            break;
        }
    }
    exit(0);  // Exit successfully
}
```

Important Notes:

- `inet_ntop()` is used to convert binary IPv4 addresses to a human-readable string.
- `h_addr_list` can contain **multiple IPs** (load-balanced servers like `cnn.com`).
- The code loops over all command-line arguments (`argv`) and resolves each hostname.

---

5. EXAMPLE OUTPUTS

Single IPv4 Address:

```sh
$ ./hostent aix
official hostname: aix.unpbook.com
address: 192.168.42.2
```

- The canonical name is fully qualified.
- One IP address is returned.

Multiple IPv4 Addresses (e.g., CNN):

```sh
$ ./hostent cnn.com
official hostname: cnn.com
address: 64.236.16.20
```

```
address: 64.236.16.52
...
```

- Indicates a **web server cluster**, where multiple IPs serve the same hostname.

Alias and Canonical Mapping (CNAME Record):

```sh
$ ./hostent www
official hostname: linux.unpbook.com
alias: www.unpbook.com
address: 206.168.112.219
```

- Shows a canonical name and alias.
- The DNS has a **CNAME record** from `www.unpbook.com` to `linux.unpbook.com`.

Errors:

```sh
$ ./hostent nosuchname.invalid
gethostbyname error for host: nosuchname.invalid: Unknown host

$ ./hostent uunet.uu.net
gethostbyname error for host: uunet.uu.net: No address associated with name
```

- First case: hostname doesn't exist.
- Second case: hostname exists but lacks an A record.

---

6. PORTABILITY AND FUTURE USE

- `gethostbyname()` is **obsolete** in modern applications.
- It is replaced by `getaddrinfo()` which:
  - Works with **IPv6**
  - Is **thread-safe**
  - Provides a more flexible structure (`struct addrinfo`)

---

7. Conclusion

This example illustrates the use of `gethostbyname()` for IPv4 DNS lookups, how the `hostent` structure organizes the DNS data (canonical name, aliases, addresses), and how to handle errors. While still functional on many systems, developers are encouraged to migrate to `getaddrinfo()` for compatibility with modern networks and to support both IPv4 and IPv6.

If you want, I can also rewrite this program using `getaddrinfo()` for a modern, robust version.

This section of the book is explaining how to use two key network-related functions in the C programming language:

1. `gethostbyaddr` – for reverse DNS lookups (IP → hostname)
2. `getservbyname` and `getservbyport` – for mapping service names ↔ port numbers

Let's go step by step.

11.4 `gethostbyaddr` Function

Purpose:

This function performs a **reverse DNS lookup**. Given a binary IP address, it tries to find the hostname that maps to that address.

Prototype:

```c
#include <netdb.h>
struct hostent *gethostbyaddr(const char *addr, socklen_t len, int family);
```

- `addr`: A pointer to an `in_addr` structure (for IPv4), cast to `char *`.
- `len`: Should be 4 for IPv4 addresses (`sizeof(struct in_addr)`).
- `family`: Should be `AF_INET` for IPv4.

Returns:

A pointer to a `hostent` structure if successful, or `NULL` on failure. If it fails, `h_errno` will be set to indicate the type of error.

DNS Note:

This function queries a **PTR record** from the DNS. PTR records are used for reverse lookups and are stored in the `in-addr.arpa` domain.

11.5 `getservbyname` and `getservbyport` Functions

Purpose:

These functions allow programmers to:

- Get the port number for a known service name (`getservbyname`)
- Get the service name for a known port number (`getservbyport`)

This allows more readable and maintainable code. Instead of hardcoding port numbers, we can use service names.

Prototype – getservbyname:

```c
#include <netdb.h>
struct servent *getservbyname(const char *servname, const char *protoname);
```

- `servname`: Name of the service (e.g., `"ftp"`, `"http"`)
- `protoname`: Protocol name (e.g., `"tcp"`, `"udp"`)

Prototype – getservbyport:

```c
#include <netdb.h>
struct servent *getservbyport(int port, const char *protoname);
```

- `port`: Must be in **network byte order** (`htons()` is used to convert from host to network order).
- `protoname`: Same as above.

Return Value:

Returns a pointer to a `servent` structure:

```c
struct servent {
    char  *s_name;      // official service name
    char **s_aliases;   // alias list
    int    s_port;      // port number (in network byte order)
    char  *s_proto;     // protocol name
};
```

Example Code Explanation: `daytimetcpcli1.c`

This code is a TCP client that connects to a daytime server and prints the time.

Let's go line by line:

```c
#include "unp.h"
```

- This likely includes standard headers, error handling, and wrapper functions (like `Socket`, `Read`, etc.)

Lines 3–4:

```c
int main (int argc, char **argv)
```

- The program expects two command-line arguments:
    1. Hostname or IP address
    2. Service name (e.g., `"daytime"`)

Lines 5–12: Declarations

```c
int sockfd, n;
char recvline[MAXLINE + 1];
struct sockaddr_in servaddr;
struct in_addr **pptr;
struct in_addr *inetaddrp[2];
struct in_addr inetaddr;
struct hostent *hp;
struct servent *sp;
```

- `sockfd`: socket file descriptor
- `recvline`: buffer to receive server's response
- `servaddr`: structure to store server's address
- `pptr`: pointer to a list of IP addresses
- `inetaddrp`: temporary array to store one IP if hostname is an IP literal
- `inetaddr`: variable to store converted IP address
- `hp`: result of hostname lookup
- `sp`: result of service name lookup

Lines 13–14: Validate Arguments

```c
if (argc != 3)
    err_quit("usage: daytimetcpcli1 <hostname> <service>");
```

- Ensure the user provided exactly two arguments.

Lines 15–26: Hostname Resolution

```c
hp = gethostbyname(argv[1])
```

- Try resolving hostname using `gethostbyname`.

- If that fails:
  - Try using `**inet_aton**` to check if it's an IP address.
  - If `**inet_aton**` fails, report error.
  - If it succeeds, use that IP to manually build a list of one address.

```c
inetaddrp[0] = &inetaddr;
inetaddrp[1] = NULL;
pptr = inetaddrp;
```

- If `**gethostbyname**` succeeds:

```c
pptr = (struct in_addr **) hp->h_addr_list;
```

- `**pptr**` now points to a list of IP addresses to try.

Line 27–28: Service Name Resolution

```c
sp = getservbyname(argv[2], "tcp")
```

- Try to resolve the service name (e.g., `**"daytime"**`) using TCP.
- On failure, exit.

Lines 29–40: Try Connecting to Each IP Address

```c
for (; *pptr != NULL; pptr++) {
    sockfd = Socket(AF_INET, SOCK_STREAM, 0);
    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_port = sp->s_port;
    memcpy(&servaddr.sin_addr, *pptr, sizeof(struct in_addr));
    printf("trying %s\n", Sock_ntop((SA *) &servaddr, sizeof(servaddr)));

    if (connect(sockfd, (SA *) &servaddr, sizeof(servaddr)) == 0)
        break; // success

    err_ret("connect error");
    close(sockfd);
}
```

- Loop through each IP address returned from DNS.
- For each address:
  - Create a socket.

- Fill in the server's socket address structure.
- Try to connect.
- If successful, break out of loop.
- If it fails, close the socket and try next.

Lines 41–42: All Connections Failed

```c
if (*pptr == NULL)
    err_quit("unable to connect");
```

- If no connection succeeded, terminate the program.

Lines 43–46: Read Server Response

```c
while ((n = Read(sockfd, recvline, MAXLINE)) > 0) {
    recvline[n] = 0; // null-terminate the string
    Fputs(recvline, stdout);
}
```

- Continuously read from the server until the connection is closed.
- Print the received time.

Line 47: Exit

```c
exit(0);
```

Example Outputs Explained

1. **Successful connection**

```nginx
freebsd % daytimetcpcli1 aix daytime
trying 192.168.42.2:13
Sun Jul 27 22:44:19 2003
```

- Connects successfully to `**aix**` on port 13 (daytime service).
- Receives current time from the server.

2. **Multihomed host with no daytime server**

```
pgsql

freebsd % daytimetcpcli1 gateway.tuc.noao.edu daytime
trying 140.252.108.1:13
connect error: Operation timed out
trying 140.252.1.4:13
connect error: Operation timed out
trying 140.252.104.1:13
connect error: Connection refused
unable to connect
```

- Tries all IP addresses returned by DNS.
- None of the servers respond or are running the daytime service.

Summary of Concepts

Function	Purpose
`gethostbyaddr`	Resolve IP → hostname
`getservbyname`	Resolve service name → port number
`getservbyport`	Resolve port number → service name
`gethostbyname`	Resolve hostname → IP address
`inet_aton`	Convert string IP to binary form
`Socket`, `Connect`	Create and establish a TCP connection
`Read`, `Fputs`	Read from socket and print to stdout

This example combines DNS resolution, service lookup, socket creation, and communication in a clean and modular way, allowing the program to be flexible across networks and configurations.

The `getaddrinfo` function is a powerful and flexible interface introduced in POSIX to address the limitations of older name resolution functions like `gethostbyname` and `gethostbyaddr`, which only supported IPv4. `getaddrinfo` supports both IPv4 and IPv6 and simplifies the process of network programming by abstracting the details of address resolution and protocol handling.

✅ Function Prototype

```c
#include <netdb.h>

int getaddrinfo(const char *hostname, const char *service,
                const struct addrinfo *hints,
                struct addrinfo **result);
```

- **Returns:** `0` on success, or a non-zero error code defined in `<netdb.h>` (see `gai_strerror` to convert it to a human-readable string).
- **Defined in:** POSIX, part of the standard networking API.

✅ Purpose of `getaddrinfo`

- Resolves hostnames and service names to one or more `sockaddr` structures.
- Supports both name-to-address and service-to-port translations.
- Replaces multiple legacy functions (`gethostbyname`, `getservbyname`, etc.).
- Abstracts protocol-specific logic so applications can use a unified interface.

✅ Parameters Explained

1. `hostname`:
   - Can be a domain name like `"example.com"` or a literal IP address.
   - If NULL and `AI_PASSIVE` is set in `hints`, binds to wildcard addresses (e.g., `INADDR_ANY` for IPv4 or `in6addr_any` for IPv6).
2. `service`:
   - Can be a service name like `"http"` or a numeric port as a string like `"80"`.
3. `hints`:
   - Optional pointer to an `addrinfo` struct that restricts the type of results returned.
   - Fields to set include:

```c
struct addrinfo {
    int              ai_flags;     // e.g., AI_PASSIVE
    int              ai_family;    // AF_INET, AF_INET6, AF_UNSPEC
    int              ai_socktype;  // SOCK_STREAM, SOCK_DGRAM
    int              ai_protocol;  // Usually 0
    socklen_t        ai_addrlen;   // Filled in by getaddrinfo
    char            *ai_canonname; // Filled in if AI_CANONNAME
    struct sockaddr *ai_addr;     // Filled in result
    struct addrinfo *ai_next;     // Next result in list
};
```

4. `result`:
   - On success, points to a **linked list** of `addrinfo` structures.
   - Must be freed with `freeaddrinfo()` after use.

✅ Common `ai_flags` Values

Flag	Description
`AI_PASSIVE`	Indicates the socket will be used for **bind** (passive socket, server).
`AI_CANONNAME`	Requests the canonical name of the host.
`AI_NUMERICHOST`	Don't resolve name; `hostname` must be numeric IP.
`AI_NUMERICSERV`	Don't resolve service name; `service` must be numeric.
`AI_V4MAPPED`	Return IPv4-mapped IPv6 addresses if no IPv6 address is found.
`AI_ALL`	Return both IPv4 and IPv6-mapped addresses.
`AI_ADDRCONFIG`	Return addresses based on local interface configuration.

✅ Typical Usage Patterns

✅ Client (TCP or UDP)

```c
struct addrinfo hints, *res;
memset(&hints, 0, sizeof(hints));
hints.ai_family = AF_UNSPEC;        // Allow IPv4 or IPv6
hints.ai_socktype = SOCK_STREAM;    // TCP socket

int err = getaddrinfo("example.com", "http", &hints, &res);
if (err != 0) {
    fprintf(stderr, "getaddrinfo error: %s\n", gai_strerror(err));
    exit(1);
}
```

- You then loop through `res`:

  ```c
  for (struct addrinfo *p = res; p != NULL; p = p->ai_next) {
      int sockfd = socket(p->ai_family, p->ai_socktype, p->ai_protocol);
      if (sockfd == -1) continue;
      if (connect(sockfd, p->ai_addr, p->ai_addrlen) == 0) {
          break; // Success
      }
      close(sockfd);
  }
  ```

✅ Server (e.g., TCP)

```c
struct addrinfo hints, *res;
memset(&hints, 0, sizeof(hints));
hints.ai_family = AF_INET;
hints.ai_socktype = SOCK_STREAM;
hints.ai_flags = AI_PASSIVE;

getaddrinfo(NULL, "8080", &hints, &res);
int sockfd = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
bind(sockfd, res->ai_addr, res->ai_addrlen);
listen(sockfd, 10);
```

✅ Multiple `addrinfo` Structures

You can receive multiple `addrinfo` nodes when:

- Host has multiple IPs.
- Service is available over TCP and UDP.
- No `ai_socktype` is specified and service is in `/etc/services`.

✅ Memory Management

- All structures returned by `getaddrinfo` are dynamically allocated.

- After use:

```c
freeaddrinfo(res);
```

✅ Canonical Name

If `AI_CANONNAME` is set, `res->ai_canonname` contains the canonical name of the host (e.g., FQDN).

✅ Summary Table: `hints` to Usage

Use Case	`hostname`	`service`	`ai_flags`	`ai_family`	`ai_socktype`
TCP Client	name/IP	port/name	0	AF_UNSPEC	SOCK_STREAM
UDP Client	name/IP	port/name	0	AF_UNSPEC	SOCK_DGRAM
TCP Server	NULL	port/name	AI_PASSIVE	AF_INET	SOCK_STREAM
UDP Server	NULL	port/name	AI_PASSIVE	AF_INET6	SOCK_DGRAM

✅ Real-World Advantages

- Protocol independence (IPv4 vs IPv6).
- Simplifies code structure.
- Supports service name lookups using `/etc/services`.
- Safer and thread-safe (unlike `gethostbyname`).

If you'd like, I can give you a full working TCP client or server code using `getaddrinfo`. Let me know your preferred language (e.g., C or C++) and context (client/server, TCP/UDP).

The passage you've provided is from a systems programming or network programming context, specifically discussing the behavior of the `getaddrinfo` and `freeaddrinfo` functions, which are used for network address resolution in C/C++ programming on POSIX systems (such as Linux, macOS, etc.).

Let me explain the content in detail, section by section.

11.8 `freeaddrinfo` Function

When you call the `getaddrinfo` function to resolve a hostname and service into an address or set of addresses, it returns a **linked list** of `struct addrinfo`. Each of these structures can contain:

- A pointer to the socket address (`ai_addr`), which itself is dynamically allocated.
- A canonical hostname (`ai_canonname`), which is also dynamically allocated if available.
- A pointer to the next `addrinfo` structure in the linked list (`ai_next`).

Because all of this memory (the `addrinfo` structures and the data they point to) is dynamically allocated, you must free it manually when done — and that's what `freeaddrinfo` is for.

Prototype:

```c
#include <netdb.h>
void freeaddrinfo(struct addrinfo *ai);
```

- `ai`: This should be a pointer to the **head of the linked list** returned by `getaddrinfo`.

When `freeaddrinfo(ai)` is called:

- It walks through the entire linked list of `addrinfo` structures.
- It **frees**:
  - The memory for each `addrinfo` node.
  - The memory for each `ai_addr`.
  - The memory for `ai_canonname`, if present.

Shallow Copy vs Deep Copy Problem

If, after calling `getaddrinfo`, you:

1. Traverse the linked list.
2. Find a `struct addrinfo` node you're interested in.
3. Copy just that node (i.e., do a shallow copy), without copying the memory it points to (like `ai_addr` and `ai_canonname`).
4. Then call `freeaddrinfo` to clean up the rest of the list.

You have a **bug**.

Why? Because the shallow copy still points to memory that `freeaddrinfo` will deallocate. After freeing, the pointers inside your copied structure will **point to invalid memory**. If you try to use those pointers (e.g., to get the address or name), you'll get undefined behavior (likely a crash or garbage data).

To do it properly, you must perform a **deep copy**, meaning:

- Allocate new memory for the structure.
- Also allocate new memory for everything it points to.
- Copy the contents of the original structure and its dependencies into your new memory.

11.9 `getaddrinfo` Function with IPv6

The `getaddrinfo` function is used for both **IPv4** and **IPv6** address resolution. It accepts various inputs and provides flexibility to the caller.

Key inputs:

- A **hints structure** (`struct addrinfo`) that tells the function what type of results the caller wants (e.g., IPv4 or IPv6).
- A **hostname** (such as `example.com` or an IP address).
- A **service name** (like `"http"` or a port number).

The returned data is a **linked list of `addrinfo` structures**, each describing one possible address.

Key Concepts from This Section

1. **Socket Address Type (Family)**

   The caller specifies the desired address family in the `hints.ai_family` field:

   - `AF_INET` → Only return IPv4 (`sockaddr_in`) addresses.
   - `AF_INET6` → Only return IPv6 (`sockaddr_in6`) addresses.
   - `AF_UNSPEC` → Return both if available.

   For example:

   - If a host has both **A (IPv4)** and **AAAA (IPv6)** DNS records, and the caller uses `AF_UNSPEC`, then:
     - The function will return a mix of `sockaddr_in` and `sockaddr_in6` structures.
     - IPv6 comes first (best practice for modern networks).

2. **Use of `AI_PASSIVE` Flag**

   This flag indicates that the caller intends to use the address in a **bind() call**, which is typical for server sockets.

   If `AI_PASSIVE` is set and no hostname is provided:

   - For IPv4, the wildcard address `0.0.0.0` is returned.
   - For IPv6, the wildcard address `::` (a.k.a. `IN6ADDR_ANY_INIT`) is returned.
   - If both are possible (e.g., `AF_UNSPEC`), both addresses may be returned.

3. **Behavior Based on Hostname Format**

   The input hostname might be:

   - A name to be resolved via DNS (e.g., `"example.com"`).
   - An IPv4 dotted-decimal string (e.g., `"192.168.1.1"`).
   - An IPv6 hexadecimal string (e.g., `"2001:db8::1"`).

   Validity of the string depends on the address family:

   - You cannot provide an IPv6 string when requesting `AF_INET` — it makes no sense.
   - Likewise, you cannot provide an IPv4 string when requesting `AF_INET6`.

   However, when using `AF_UNSPEC`, both formats are accepted.

4. **IPv4-mapped IPv6 Addresses**

   These are special IPv6 addresses that embed an IPv4 address in IPv6 space, using the format `::ffff:<IPv4>` (e.g., `::ffff:192.168.1.1`).

If the caller wants only IPv6 addresses but still wants to use IPv4 addresses, they can set:

- `AI_V4MAPPED` — to return IPv4 addresses as IPv4-mapped IPv6.
- `AI_ALL` — to get both IPv6 and IPv4-mapped IPv6.

But the spec says this conversion should not happen automatically when both A and AAAA records are available, since both can be returned natively (with `AF_UNSPEC`).

5. **Order of Results**

   On a dual-stack machine (which supports both IPv4 and IPv6), it is recommended to return the **IPv6 wildcard address first**. That's because a socket bound to the IPv6 wildcard address can, on some systems, accept connections from both IPv4 and IPv6 clients.

Conclusion

- Always use `freeaddrinfo` after `getaddrinfo` to prevent memory leaks.
- Avoid shallow copies of `addrinfo` structures; use deep copies if you want to store an entry.
- Be mindful of the `ai_family`, `ai_flags`, and the format of the hostname you provide.
- The combination of `AF_UNSPEC`, `AI_V4MAPPED`, `AI_ALL`, and whether the hostname is NULL or not can lead to varied and powerful behaviors for resolving addresses.
- `getaddrinfo` is designed to handle both IPv4 and IPv6 gracefully, and understanding how it handles wildcard addresses, DNS records, and mapped addresses is essential for writing robust networked applications.

Let me know if you'd like Figure 11.8 (referred to in the text) to be recreated or explained in detail as well.

This section (11.10 and 11.11) builds on your understanding of the `getaddrinfo` function from the POSIX API for network programming. It provides practical usage examples and introduces a helper function `host_serv` to simplify its usage.

Section 11.10: `getaddrinfo` Function: Examples

Context Recap

`getaddrinfo` is a modern, protocol-independent function to convert hostnames and service names into a list of `addrinfo` structures, each of which can be used to create a socket. It's the preferred replacement for older functions like `gethostbyname` and `getservbyname`.

The examples in this section use a test program (`testga`) that takes various command-line arguments to demonstrate how different combinations of parameters affect the behavior and output of `getaddrinfo`.

Example 1:

```bash
testga -f inet -c -h freebsd4 -s domain
```

- `-f inet`: Use IPv4 (`AF_INET`)
- `-c`: Request the canonical name (`AI_CANONNAME`)
- `-h freebsd4`: Hostname is `freebsd4`
- `-s domain`: Service is `domain` (typically UDP port 53 for DNS)

Output:

```text
socket (AF_INET, SOCK_DGRAM, 17), ai_canonname = freebsd4.unpbook.com
address: 135.197.17.100:53
socket (AF_INET, SOCK_DGRAM, 17)
address: 172.24.37.94:53
socket (AF_INET, SOCK_STREAM, 6), ai_canonname = freebsd4.unpbook.com
address: 135.197.17.100:53
socket (AF_INET, SOCK_STREAM, 6)
address: 172.24.37.94:53
```

Explanation:

- Two socket types are returned: `SOCK_DGRAM` (UDP) and `SOCK_STREAM` (TCP) for each of the IP addresses.
- Both UDP and TCP support the DNS `domain` service.
- Multiple IP addresses mean the host is multihomed (multiple interfaces or addresses).

Example 2:

```bash
testga -f inet -t stream -h gateway.tuc.noao.edu -s daytime
```

- `-t stream`: Restrict to TCP sockets (`SOCK_STREAM`)
- Three addresses are returned for the host (again, multihomed)
- All with port 13, which corresponds to the `daytime` service

Output:

```text
socket (AF_INET, SOCK_STREAM, 6)
address: 140.252.108.1:13
...
```

Example 3:

```bash
testga -h aix -s ftp -t stream
```

- No address family specified, so `AF_UNSPEC` is assumed
- Host `aix` has both AAAA (IPv6) and A (IPv4) records

Output:

```text
socket (AF_INET6, SOCK_STREAM, 6)
address: [3ffe:b80:...]:21
socket (AF_INET, SOCK_STREAM, 6)
address: 192.168.42.2:21
```

Explanation:

- Two records are returned, one for IPv6 and one for IPv4
- Port 21 is used for FTP (default for TCP)

Example 4:

```bash
testga -p -s 8888 -t stream
```

- `-p`: AI_PASSIVE flag is set, used for server-side code that binds to any address
- No hostname: wildcard address is assumed
- No family specified: both IPv6 and IPv4 are returned

Output:

```text
socket (AF_INET6, SOCK_STREAM, 6)
address: [::]:8888
socket (AF_INET, SOCK_STREAM, 6)
address: 0.0.0.0:8888
```

Explanation:

- The wildcard addresses returned are suitable for binding and accepting connections on all interfaces.
- IPv6 comes before IPv4 because dual-stack IPv6 servers can accept both IPv6 and IPv4 clients using IPv4-mapped addresses.

Section 11.11: `host_serv` Function

Purpose

The `host_serv` function is a wrapper around `getaddrinfo` designed to simplify its use. Instead of forcing the caller to fill in a `hints` structure every time, it encapsulates that logic.

Code:

```c
#include "unp.h"

struct addrinfo *
host_serv(const char *host, const char *serv, int family, int socktype)
{
    int n;
    struct addrinfo hints, *res;

    bzero(&hints, sizeof(struct addrinfo));
    hints.ai_flags = AI_CANONNAME;     // Always ask for canonical name
    hints.ai_family = family;          // AF_INET, AF_INET6, AF_UNSPEC
    hints.ai_socktype = socktype;      // SOCK_STREAM, SOCK_DGRAM, etc.

    if ((n = getaddrinfo(host, serv, &hints, &res)) != 0)
        return NULL;

    return res; // Return pointer to the first addrinfo in the linked list
}
```

Explanation Line-by-Line:

- `bzero(&hints, sizeof(...))`: Initializes the `hints` structure to zero.
- `hints.ai_flags = AI_CANONNAME`: Requests the canonical name of the host (fully qualified domain name).
- `hints.ai_family`: Sets the desired address family (can be `AF_INET`, `AF_INET6`, or `AF_UNSPEC`).
- `hints.ai_socktype`: Specifies socket type, e.g., `SOCK_STREAM` for TCP or `SOCK_DGRAM` for UDP.
- `getaddrinfo(...)`: Calls the system resolver to get address info based on the specified hints.
- If successful, returns a linked list of `addrinfo` structures; otherwise returns `NULL`.

Key Advantages of `host_serv`:

1. **Cleaner API:** No need for user to understand or manage the `hints` struct.
2. **Canonical name included:** Useful when logging or verifying DNS info.
3. **Handles memory allocation internally:** Caller only needs to free it using `freeaddrinfo()`.

Conclusion

Together, these two sections provide both:

- **Practical insight** into how `getaddrinfo` behaves with various configurations (e.g., multiple IPs, IPv6, wildcard addresses)

- **Convenience abstraction** via `host_serv` for simplifying use of `getaddrinfo` in client-server applications

This is essential for modern socket programming where dual-stack (IPv4 + IPv6) compatibility and DNS-based flexibility are expected.